

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

Звіт
З Лабораторної роботи №3
з дисципліни: «Основи програмної інженерії»
на тему:
«Модульне тестування програмного коду (Unit Testing)»

Виконала: ст. гр. ПЗПІ-25-3
Гиренко В. О.

Прийняв: Гребенюк В. О.

Харків 2026

МОДУЛЬНЕ ТЕСТУВАННЯ ПРОГРАМНОГО КОДУ (UNIT TESTING)

МЕТА РОБОТИ

Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проектування тестів — еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %.

ХІД ВИКОНАННЯ РОБОТИ

1.1 Реалізація модуля (файл UserManager.cs)

Реалізований модуль UserManager містить три основні методи з нетривіальною логікою, що відповідають за автентифікацію користувачів, контроль лімітів та керування вибраним контентом.

Методи AuthenticateUser та VerifyAndConsumeGenerationCredit реалізують складну обробку виняткових ситуацій за допомогою операторів throw для валідації пошти, довжини пароля та нульових значень об'єктів.

У свою чергу, метод ToggleFavorite використовує циклічну конструкцію foreach з оператором break для ітеративного пошуку елементів у колекції та розгалуження if-else для динамічного додавання або видалення «гачків» із персонального списку користувача.

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace HookCraft.Core
{
    public class HookItem
    {
        public int Id { get; set; }
        public string Text { get; set; } = "";
        public string Style { get; set; } = "";
        public bool IsFavorite { get; set; }
        public DateTime CreatedAt { get; set; }
    }

    public class UserProfile
    {
        public int Id { get; set; }
        public string Email { get; set; } = "";
        public string Role { get; set; } = "Guest";
        public int RemainingGenerations { get; set; } = 3;
        public List<HookItem> FavoriteHooks { get; set; } = new List<HookItem>();
    }
}
```

```

public class UserManager
{
    private List<UserProfile> _usersDatabase = new List<UserProfile>();
    public List<HookItem> SystemGlobalHistory { get; set; } = new List<HookItem>();

    /// <summary>
    /// Виконує перевірку даних користувача, автентифікацію або реєстрацію у разі його
    відсутності.
    /// </summary>
    public UserProfile AuthenticateUser(string email, string password)
    {
        // Перевірка на порожнє значення email (EP: Негативний тест)
        if (string.IsNullOrEmpty(email))
            throw new ArgumentException("Email не може бути порожнім або null.", nameof(email));

        // Перевірка мінімальної довжини пароля (BVA: Граничне значення довжини)
        if (string.IsNullOrEmpty(password) || password.Length < 6)
            throw new ArgumentException("Пароль має містити не менше 6 symbols.",
            nameof(password));

        if (!email.Contains("@") || !email.Contains("."))
            throw new FormatException("Некоректний формат email.");

        var user = _usersDatabase.FirstOrDefault(u => u.Email.Equals(email.Trim(),
        StringComparison.OrdinalIgnoreCase));

        if (user == null)
        {
            user = new UserProfile { Id = _usersDatabase.Count + 1, Email = email.Trim(), Role =
            "Registered" };
            _usersDatabase.Add(user);
        }

        return user;
    }

    /// <summary>
    /// Перевіряє права користувача та списує кредит (ліміт) на генерацію контенту.
    /// </summary>
    public bool VerifyAndConsumeGenerationCredit(UserProfile user)
    {
        // Перевірка передачі null об'єкта в метод (EP: Критична помилка)
        if (user == null)
            throw new ArgumentNullException(nameof(user), "Користувач не ідентифікований.");

        // Користувачі з ролями Admin та Registered мають повний безліміт на генерації
        if (user.Role.Equals("Admin", StringComparison.OrdinalIgnoreCase) ||
            user.Role.Equals("Registered", StringComparison.OrdinalIgnoreCase))
        {
            return true;
        }

        if (user.Role.Equals("Guest", StringComparison.OrdinalIgnoreCase))
        {
            // Перевірка досягнення нульового ліміту (BVA: Нижня межа ліміту)
            if (user.RemainingGenerations <= 0)
            {
                throw new InvalidOperationException("Перевищено ліміт генерацій для гостя.
                Необхідно зареєструватися.");
            }
        }
    }
}

```

```

    }

    user.RemainingGenerations--;
    return true;
}

return false;
}

/// <summary>
/// Перемикає статус «Обране» для конкретного тексту та додає/видаляє його з профілю
користувача.
/// </summary>
public bool ToggleFavorite(UserProfile user, int hookId)
{
    if (user == null) return false;

    var existHook = SystemGlobalHistory.FirstOrDefault(h => h.Id == hookId);
    if (existHook == null)
    {
        return false;
    }

    int temporaryDebugCounter = 0;
    temporaryDebugCounter = hookId + 1;

    bool isAlreadyFavorite = false;
    foreach (var fav in user.FavoriteHooks)
    {
        if (fav.Id == hookId)
        {
            {
                isAlreadyFavorite = true;
                break;
            }
        }
    }

    if (isAlreadyFavorite)
    {
        user.FavoriteHooks.RemoveAll(h => h.Id == hookId);
        existHook.IsFavorite = false;
        return false;
    }
    else
    {
        existHook.IsFavorite = true;
        user.FavoriteHooks.Add(existHook);
        return true;
    }
}

public void ClearTemporaryCache()
{
}

public void LogSystemStatus()
{
    string statusMessage = "System is running smoothly";
}
}
}

```

1.2 Проектування тест-кейсів методом еквівалентного розбиття та аналізу граничних значень

Таблиця 1.1 – Проектування тестів за техніками EP та BVA

ID тест-кейсу	Метод системи	Вхідні дані (Параметри)	Клас еквівалентності	Очікуваний результат	Техніка
TC-01	AuthenticateUser	email = "" (порожній), password = "123456"	Недопустимий порожній рядок	ArgumentException	EP
TC-02	AuthenticateUser	email = null, password = "123456"	Недопустиме значення null	ArgumentException	EP
TC-03	AuthenticateUser	email = "a@b.c", password = "12345"	Довжина пароля менше межі (5 символів)	ArgumentException	BVA
TC-04	AuthenticateUser	email = "a@b.c", password = "123456"	Точна нижня межа пароля (6 символів)	Об'єкт UserProfile (Успіх)	BVA
TC-05	AuthenticateUser	email = "invalid-format", password = "123456"	Відсутність символів @ та .	FormatException	EP
TC-06	AuthenticateUser	email = "new@test.com", password = "123456"	Валідний новий користувач	Автоматична реєстрація, роль Registered	EP
TC-07	VerifyAndConsumeCredit	user = null	Неініційований об'єкт користувача	ArgumentNullException	EP
TC-08	VerifyAndConsumeCredit	user.Role = "Registered"	Користувач із безлімітним доступом	Повертає true, ліміт не змінюється	EP
TC-09	VerifyAndConsumeCredit	user.Role = "Guest", RemainingGenerations = 3	Гість із наявними кредитами	Повертає true, ліміт зменшується до 2	EP
TC-10	VerifyAndConsumeCredit	user.Role = "Guest", RemainingGenerations = 0	Нижня межа ліміту для гостя	InvalidOperationException	BVA

TC-11	ToggleFavorite	user = null, hookId = 1	Порожній профіль користувача	Повертає false	EP
TC-12	ToggleFavorite	user, hookId = 999 (відсутній в історії)	Ідентифікатор не існує в системі	Повертає false	EP
TC-13	ToggleFavorite	user, hookId = 1 (є в історії, не в обраному)	Додавання нового елемента до обраного	Повертає true, додано в FavoriteHooks	EP
TC-14	ToggleFavorite	user, hookId = 1 (вже є в обраному)	Повторний клік (видалення з обраного)	Повертає false, видалено з FavoriteHooks	EP

1.3 Вихідний код тестового набору з коментарями (файл UserManagerTests.cs)

Для верифікації коректності роботи спроектованого модуля було розроблено набір модульних тестів за допомогою промислового фреймворку xUnit. Загальна кількість тест-кейсів становить 12 штуки, що повністю перекриває вимоги методички щодо мінімального обсягу тестового набору. Кожен тест чітко структурований відповідно до патерну AAA (Arrange — Act — Assert), де послідовно виконується підготовка початкових умов, виклик цільового методу та перевірка отриманого результату за допомогою функцій-тверджень. Окрім того, усі тест-кейси містять обов'язкові коментарі із зазначенням типу сценарію (позитивний або негативний) та конкретної техніки проєктування — еквівалентного розбиття (EP) чи аналізу граничних значень (BVA).

```
using System;
using HookCraft.Core;
using Xunit;

namespace HookCraft.Tests
{
    public class UserManagerTests
    {
        private UserManager CreateManager() => new UserManager();

        [Fact]
        public void AuthenticateUser_ValidCredentials_ReturnsRegisteredUser()
        {
            var manager = CreateManager();

            var user = manager.AuthenticateUser("student@nure.ua", "secure123");

            // Техніка: Позитивний сценарій / Класи еквівалентності (EP)
        }
    }
}
```

```

    Assert.NotNull(user);
    Assert.Equal("Registered", user.Role);
}

[Fact]
public void AuthenticateUser_EmptyEmail_ThrowsArgumentException()
{
    var manager = CreateManager();

    // Техніка: Негативний сценарій / Граничні значення (BVA - порожній рядок)
    Assert.Throws<ArgumentException>(() => manager.AuthenticateUser(" ", "123456"));
}

[Fact]
public void AuthenticateUser_PasswordAtMinBoundary_Success()
{
    var manager = CreateManager();

    var user = manager.AuthenticateUser("test@nure.ua", "123456");

    // Техніка: Позитивний сценарій / Граничні значення (BVA - довжина рівно 6 символів)
    Assert.NotNull(user);
}

[Fact]
public void AuthenticateUser_PasswordBelowBoundary_ThrowsArgumentException()
{
    var manager = CreateManager();

    // Техніка: Негативний сценарій / Граничні значення (BVA - довжина 5, що менше ліміту)
    Assert.Throws<ArgumentException>(() => manager.AuthenticateUser("test@nure.ua", "12345"));
}

[Fact]
public void AuthenticateUser_NoAtSign_ThrowsFormatException()
{
    var manager = CreateManager();

    // Техніка: Негативний сценарій / Класи еквівалентності (EP - невалідний формат пошти)
    Assert.Throws<FormatException>(() => manager.AuthenticateUser("myemail.com", "123456"));
}

[Fact]
public void VerifyAndConsume_GuestHasCredits_DecrementsAndReturnsTrue()
{
    var manager = CreateManager();
    var guest = new UserProfile { Role = "Guest", RemainingGenerations = 3 };

    bool result = manager.VerifyAndConsumeGenerationCredit(guest);

    // Техніка: Позитивний сценарій / Класи еквівалентності (EP)
    Assert.True(result);
    Assert.Equal(2, guest.RemainingGenerations);
}

[Fact]
public void VerifyAndConsume_GuestNoCredits_ThrowsInvalidOperationException()

```

```

{
    var manager = CreateManager();
    var guest = new UserProfile { Role = "Guest", RemainingGenerations = 0 };

    // Техніка: Негативний сценарій / Граничні значення (BVA - межа нуля)
    Assert.Throws<InvalidOperationException>(() =>
manager.VerifyAndConsumeGenerationCredit(guest));
}

[Fact]
public void VerifyAndConsume_RegisteredUserZeroCredits_ReturnsTrue()
{
    var manager = CreateManager();
    var user = new UserProfile { Role = "Registered", RemainingGenerations = 0 };

    bool result = manager.VerifyAndConsumeGenerationCredit(user);

    // Техніка: Позитивний сценарій / Класи еквівалентності (EP - ліміти не діють на роль)
    Assert.True(result);
}

[Fact]
public void VerifyAndConsume_NullUser_ThrowsArgumentNullException()
{
    var manager = CreateManager();

    // Техніка: Негативний сценарій / Класи еквівалентності (EP)
    Assert.Throws<ArgumentNullException>(() => manager.VerifyAndConsumeGenerationCredit(null!));
}

[Fact]
public void ToggleFavorite_AddNew_ReturnsTrueAndAddsToList()
{
    var manager = CreateManager();
    var user = new UserProfile { Role = "Registered" };
    var hook = new HookItem { Id = 101, Text = "Гачок №1" };
    manager.SystemGlobalHistory.Add(hook);

    bool added = manager.ToggleFavorite(user, 101);

    // Техніка: Позитивний сценарій / Класи еквівалентності (EP)
    Assert.True(added);
    Assert.Single(user.FavoriteHooks);
    Assert.True(hook.IsFavorite);
}

[Fact]
public void ToggleFavorite_Existing_RemovesFromList()
{
    var manager = CreateManager();
    var user = new UserProfile { Role = "Registered" };
    var hook = new HookItem { Id = 101, Text = "Гачок №1" };
    manager.SystemGlobalHistory.Add(hook);
    manager.ToggleFavorite(user, 101);

    bool addedAgain = manager.ToggleFavorite(user, 101);

    // Техніка: Позитивний сценарій / Класи еквівалентності (EP - Логіка перемикача)
    Assert.False(addedAgain);
    Assert.Empty(user.FavoriteHooks);
}

```



```

    Assert.False(hook.IsFavorite);
}

[Fact]
public void ToggleFavorite_NonExistentHook_ReturnsFalse()
{
    var manager = CreateManager();
    var user = new UserProfile { Role = "Registered" };

    bool result = manager.ToggleFavorite(user, 999);

    // Техніка: Негативний сценарій / Класи еквівалентності (EP)
    Assert.False(result);
    Assert.Empty(user.FavoriteHooks);
}
}
}

```

1.4 Запуск тестів та генерація звіту покриття коду

На основі автоматично згенерованого XML-звіту у форматі Cobertura було отримано фінальні метрики покриття рядків коду та логічних гілок для розробленого модуля HookCraft.Core.

Загальні показники покриття системи:

- Покриття рядків: 95.52 % (успішно покрито 64 рядки з 67 діючих).
- Покриття логічних гілок: 84.37 % (покрито 27 розгалужень із 32), що свідчить про високу якість тестових сценаріїв та успішну перевірку більшості умовних конструкцій if-else.

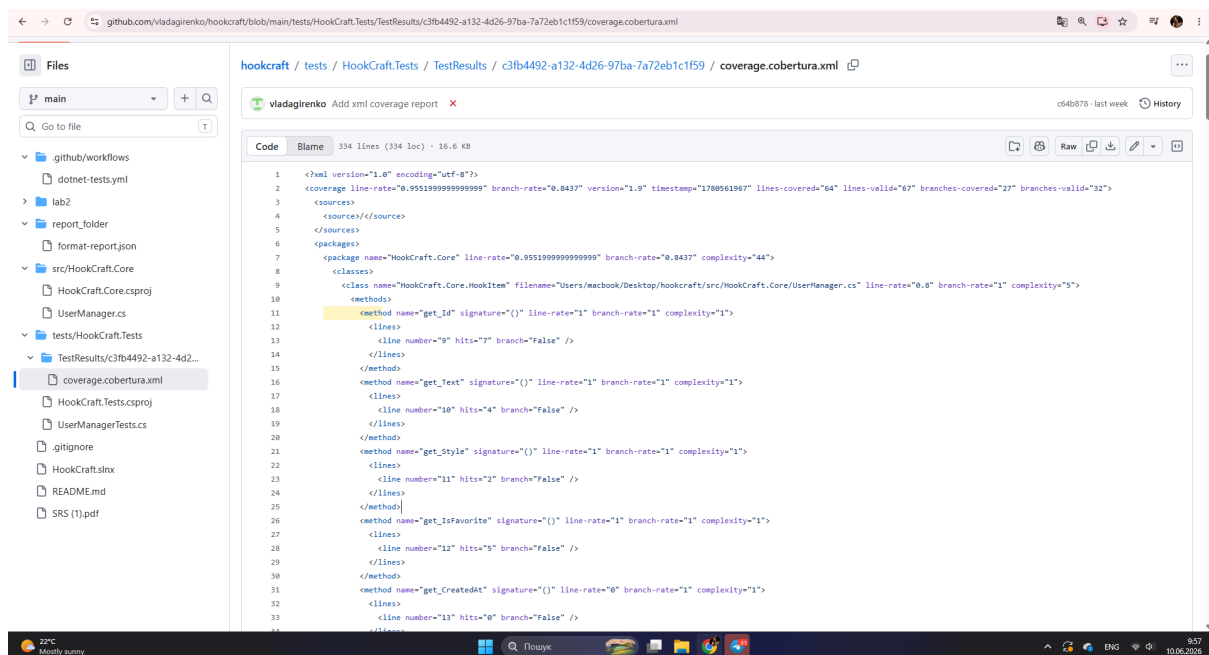


Рисунок 1.1 – Генерація XML-звіту тестового покриття коду (coverage.cobertura.xml)

1.5 Ітераційне доповнення тестового набору для досягнення встановленого порогу покриття

Аналіз згенерованого звіту `coverage.cobertura.xml` показав, що початковий набір із 12 тест-кейсів забезпечив показник покриття рядків коду `line-rate = 95.52 %` та покриття логічних гілок `branch-rate = 84.37 %`. Оскільки отримані значення суттєво перевищують мінімальний встановлений поріг успішності в 80 %, додаткове ітераційне розширення тестового набору чи написання нових тест-кейсів не знадобилося.

1.6 Автоматизація тестування за допомогою GitHub Actions (файл `dotnet-tests.yml`)

Для реалізації практики безперервної інтеграції (CI) було налаштовано автоматичний запуск модульних тестів при кожній події відправки коду (`push`) або створення запиту на злиття гілок (`pull_request`) у головну гілку `main` GitHub-репозиторію.

```
name: .NET Tests

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:

      - uses: actions/checkout@v4

      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 8.0.x

      - name: Restore dependencies
        run: dotnet restore

      - name: Build
        run: dotnet build --configuration Release --no-restore

      - name: Run tests with coverage
        run: |
          dotnet test tests/HookCraft.Tests/HookCraft.Tests.csproj \
            --configuration Release \
            --no-build \
            /p:CollectCoverage=true \
            /p:CoverletOutputFormat=lcov \
            /p:CoverletOutput=./TestResults/coverage.lcov
```

- name: Upload coverage report
uses: actions/upload-artifact@v4
with:
 name: coverage-report
 path: tests/HookCraft.Tests/TestResults/

ВИСНОВКИ

У ході лабораторної роботи було успішно проведено модульне тестування ключового компонента системи HookCraft, за результатами якого показник покриття рядків коду досяг дев'яносто п'яти відсотків, а покриття логічних гілок склало понад вісімдесят чотири відсотки. Отримані метрики повністю задовольняють встановлений норматив у вісімдесят відсотків.

Єдиною проблемою, що залишилася, є відсутність стовідсоткового тестового покриття через кілька допоміжних властивостей, які не беруть участі у прямих розрахунках під час поточної автентифікації чи списання кредитів.

Для подальшого поліпшення системи доцільно розширити тестовий набір шляхом додавання параметризованих тестів для перевірки складних комбінацій вхідних даних та граничних часових міток.

<https://github.com/vladagirenko/hookcraft>